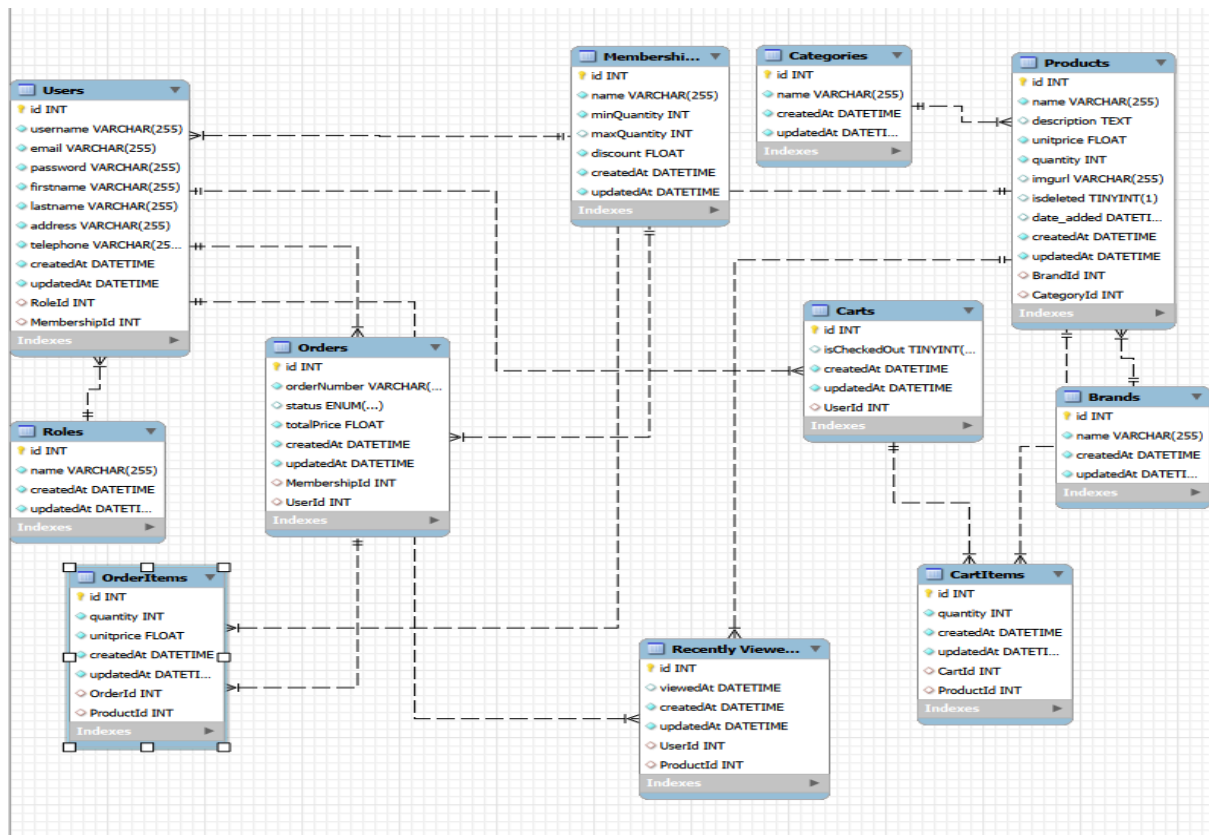


Reflection Report – EP Marketplace

Planning and Foundation:

My first week started slower due to the 17th May, but I still got a decent amount done. I cleaned up the Express boilerplate, designed the schema in 3NF, and set up a hosted MySQL instance on Railway. Hosting the DB from the start was a smart call - my production environment was ready early and I didn't hit surprises during deployment. Drawing the ERD before coding a single model saved me from structural mistakes that would've been painful to fix later. Setting up Jira with Epics and Stories was as frustrating as usual, but it kept remaining tasks visible at every point.



Backend Development:

The backend runs on Node.js, Express, Sequelize, and MySQL on Railway. The database is in 3NF with eleven tables: Users, Roles, Membership, Products, Categories, Brands, Orders, OrderItems, Carts, CartItems, and RecentlyViewed, all linked through Sequelize associations. For auth I used JWT with Bcrypt for password hashing. Two roles - Admin and User - are stored in separate Roles tables; `auth.middleware.js` checks the JWT on every request and `admin.middleware.js` confirms the Admin role.

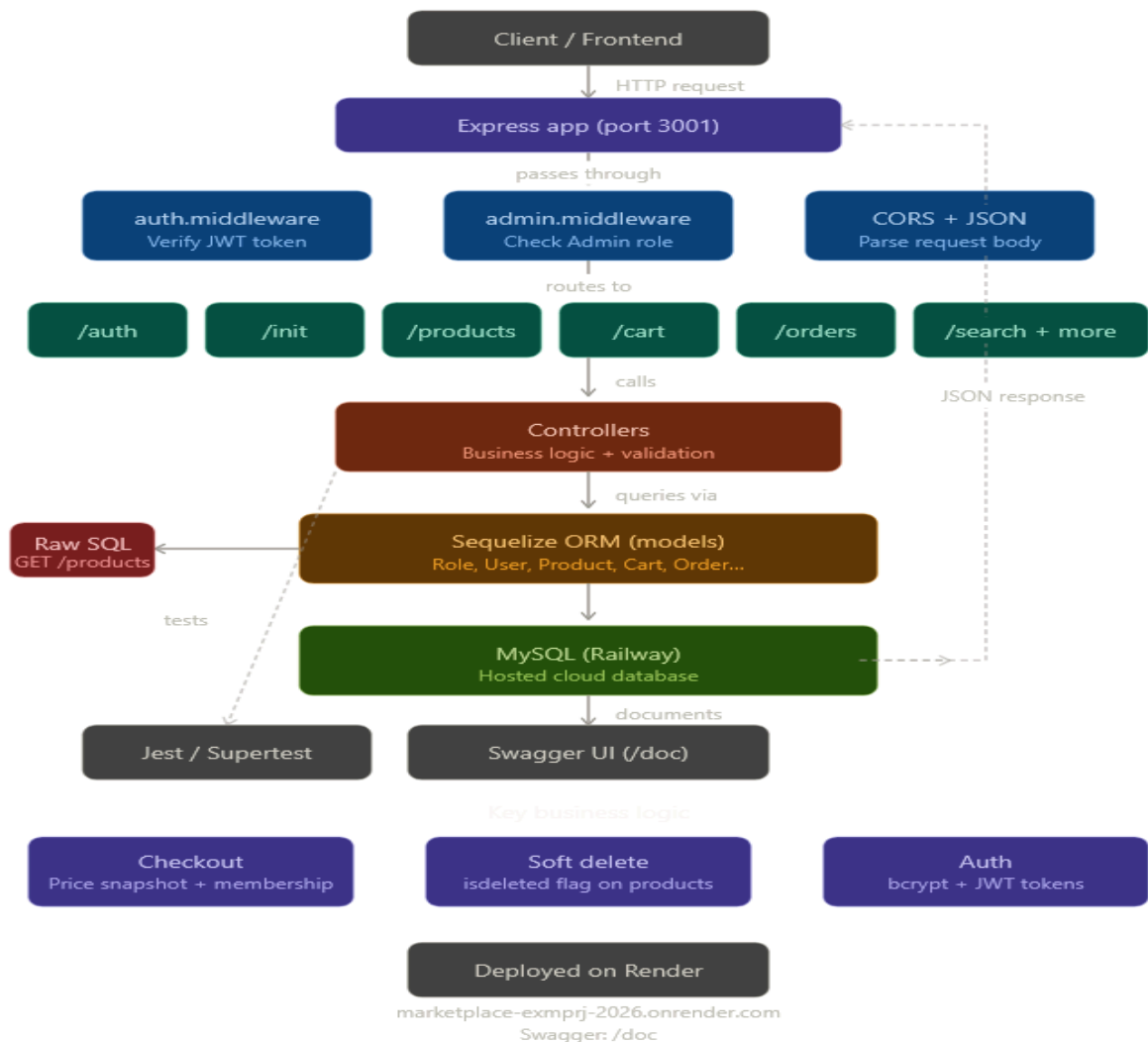
I locked in some architectural decisions early; soft delete on products via an `isDeleted` flag; a price snapshot on `OrderItems` storing `unitPrice` at checkout so old orders stay accurate even if prices change later, automatic membership tier upgrades after checkout based on cumulative items purchased; quantity incrementing on duplicate cart rows via `CartItem.increment()` instead of inserting a new row; and order numbers generated as 8-character strings using `Math.random()` and `Number.toString()`.

Three bugs took real time to track down. First, `sequelize.sync({ alter: true })` tried adding too many indexes to the Users table and hit MySQL's 64-key limit - fixed by switching to `{ force: false }`. Second, I'd written `header.split('')[1]` with an empty string instead of a space, so every token extraction returned `undefined` and quietly broke all authenticated requests. Third, `OrderItem.sum('unitPrice', { include: [...] })` kept failing under MySQL's `only_full_group_by` mode because Sequelize's generated SQL didn't satisfy the grouping constraint - I replaced it with raw SQL using `sequelize.query('SELECT SUM(unitPrice) FROM OrderItems WHERE ...')`, which was a useful lesson in when the ORM abstraction should be abandoned.

Checkout was the most complex function - it had to calculate the total with membership discounts, generate a unique order number, snapshot unit prices, reduce stock, and upgrade the membership tier, all in sequence. The discount logic is intentional: the rate applied is based on the membership at the time of checkout, and the upgrade fires afterward. So a user on the cusp of Bronze and Silver pays the Bronze rate on the qualifying order and benefits from Silver on the next.

For `GET /products` and `POST /search` I used parameterized queries via Sequelize's `replacements` feature rather than interpolating user input directly into SQL strings. This was the first time I properly understood why string interpolation in SQL is dangerous - if user input isn't sanitized, an attacker can break out of the query and execute arbitrary SQL injection to the database. All other endpoints use standard CRUD methods like `Product.findAll()`, `Product.create()`, and `Product.update()`. A `POST /init` endpoint seeds the database with roles, an admin user, membership tiers, and products fetched from the Noroff API.

The API is documented with Swagger UI at `/doc`, set up early so the JSDoc comments above each route doubled as an endpoint checklist during development. For testing I used Jest and Supertest with integration tests that hit the API end to end. The main issue was that `sequelize.sync()` was called in `app.js`, and since Jest imports `app.js` directly, the sync ran on every test import — causing timing issues and leaving connections open. I moved `sync()` to `bin/www` so it only runs when the server actually starts, and used `afterAll(() => sequelize.close())` to close the connection cleanly. All eight tests pass. Deploying to Render needed one last fix: `Role.js` had to be committed with a lowercase filename.



Frontend Development:

The frontend is a separate React app built with Vite, deployed to Vercel, communicating with the backend only through Axios — no direct database connection. I chose Vite over Next.js because the backend was already in place and I didn't need server-side rendering. One annoying snag: `npm install tailwindcss` installed Tailwind v4, which had moved the PostCSS plugin — fixed by explicitly installing `tailwindcss@3`.

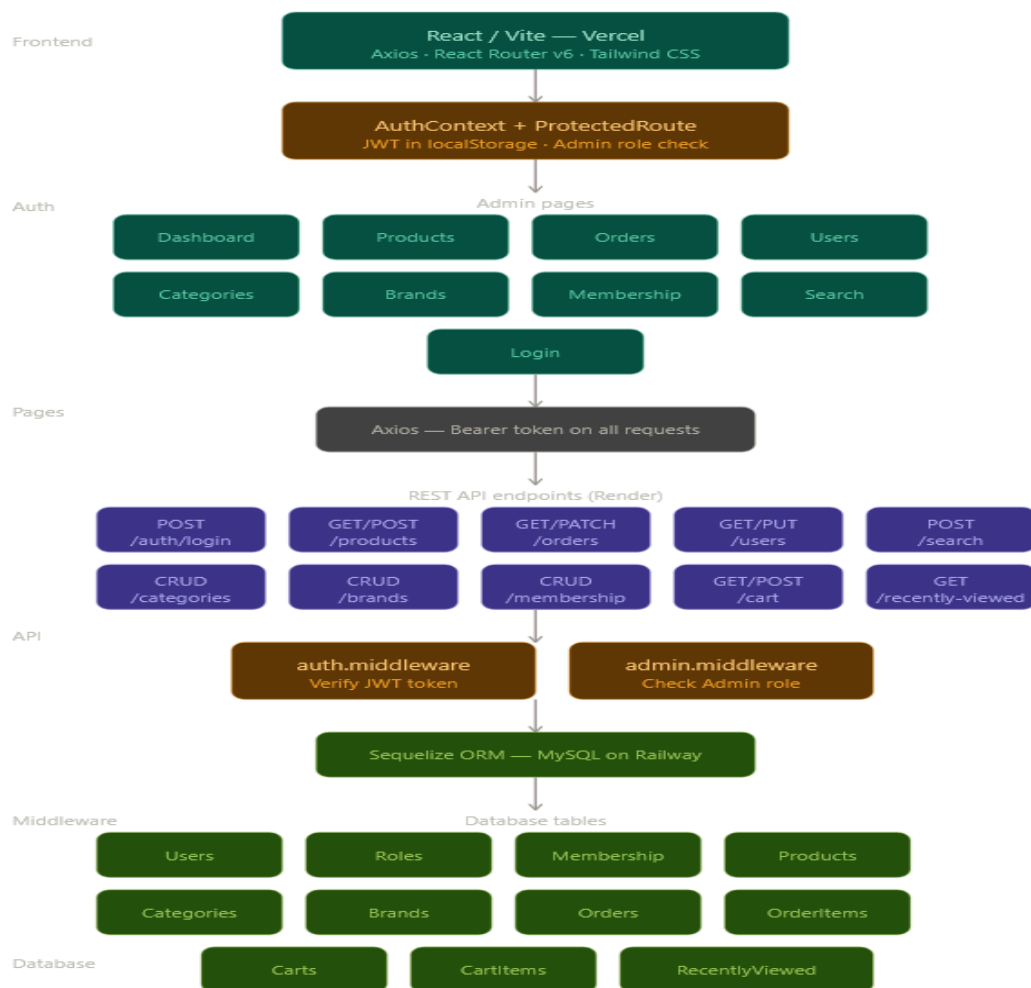
I split `src/` into `api/`, `context/`, `components/`, and `pages/`. A central `axiosInstance.js` reads the base URL from `VITE_API_URL`, attaches the JWT from `localStorage` to every request via an interceptor, and handles 401 responses by clearing storage and redirecting to `/login`. Global auth state lives in `AuthContext.jsx` using the Context API, rehydrating `user` and `token` from `localStorage` on mount so users stay logged in after a refresh. Protected routes are wrapped in `ProtectedRoute`, which checks

both the token and the Admin role. A `vercel.json` catch-all rewrite prevents 404s on hard refresh.

I built reusable components for modals (`Modal.jsx` with `ModalField` and `ModalFooter` named exports), badges (`Badge.jsx`), stat cards (`StatCard.jsx`), the sidebar (`Sidebar.jsx`), and the shared layout (`Layout.jsx`). Toast notifications run through `ToastContext.jsx` and auto-dismiss after 3.5 seconds. Nine pages were built in total, each following the same state pattern of local `useState` for data and loading/error flags.

The fiddly part was matching backend field names exactly: `date_added` instead of `createdAt`, `totalPrice` instead of `total`, `order.User.username` as a nested object rather than a flat field, and `RoleId` as an integer. These small mismatches caused silent failures that were disproportionately time-consuming to debug.

For the dashboard I added charts via `recharts`: a bar chart showing product count by category, a line chart showing revenue across recent orders, and widgets for brands and membership tiers. The category chart is built dynamically by filtering the products array against the categories list, so it updates automatically as data changes.



Challenges and Learnings:

The most useful learning came when abstractions broke down - when Sequelize diverged from expectations or MySQL's strict mode forced me into raw SQL. Those moments required actual research and produced more durable understanding than anything that just worked first time.

The most time-consuming part of the whole project wasn't writing code - it was keeping the backend's data shapes consistent with what the frontend expected. If I had to do it again, I'd write the tests earlier; they would have caught the middleware split bug and the `sync()` timing issue much sooner.

References:

Backend

Packages:

- **cookie-parser** — Middleware for parsing cookies in Express requests.
<https://www.npmjs.com/package/cookie-parser>

Sequelize:

- **Model synchronization** — How `sequelize.sync()` works, including `force` and `alter` options and why `alter: true` can be destructive.
<https://sequelize.org/docs/v7/models/model-synchronization/>
- **Model querying — finders** — Core querying methods: `findAll()`, `findOne()`, `findByPk()`, and query options like `where`, `include`, and `order`.
<https://sequelize.org/docs/v6/core-concepts/model-querying-finders/>
- **findOne** — Returns the first row matching a `where` clause; used for single-record lookups like login and cart checks.
<https://sequelize.org/docs/v6/core-concepts/model-querying-finders/#findone>
- **Raw queries (v6)** — How to run raw SQL through Sequelize using `sequelize.query()` with `replacements` for parameterized input.
<https://sequelize.org/docs/v6/core-concepts/raw-queries/>

- **Raw queries (v7)** — Updated raw query API; referenced for cross-version consistency. <https://sequelize.org/docs/v7/querying/raw-queries/>
- **Model.upsert upgrade notes** — Documents breaking changes to upsert behavior between Sequelize versions. <https://sequelize.org/docs/v7/other-topics/upgrade/#modelupsert>
- **GitHub issue — sync alter behavior** — Community discussion on why `alter: true` causes unexpected index behavior and how to work around it. <https://github.com/sequelize/sequelize/issues/3256>

MySQL:

- **GROUP BY handling (only_full_group_by)** — Explains MySQL's strict grouping mode, which requires all non-aggregated columns to be explicitly grouped. Used to understand why `OrderItem.sum()` with an include failed and why raw SQL was the fix. <https://dev.mysql.com/doc/refman/8.4/en/group-by-handling.html>

JavaScript (MDN / javascript.info):

- **String methods** — Reference for string operations including `split()`, which was central to the auth middleware bug where an empty string separator broke token extraction. <https://javascript.info/string#methods>
- **Greater than operator (>)** — Used in membership tier boundary comparisons during checkout logic. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Greater_than
- **Number.toFixed()** — Rounds a number to a fixed number of decimal places; used for formatting prices and totals. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toFixed
- **parseFloat()** — Converts string input to a floating-point number; used when handling price values from the API or request body. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/parseFloat
- **Math.random()** — Generates a pseudo-random number between 0 and 1; used as part of the 8-character order number generation. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random

- **Number.toString(radix)** — Converts a number to a string in a given base; called with `36` to produce alphanumeric order numbers from `Math.random()`.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toString
- **String.substring()** — Extracts a portion of a string by index; used to trim the order number to exactly 8 characters.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/substring

Security:

- **Node.js SQL injection guide** — Explains how SQL injection works in Node.js, with examples of vulnerable patterns and how parameterized queries prevent them.
<https://www.stackhawk.com/blog/node-js-sql-injection-guide-examples-and-prevention/>
- **Preventing SQL injection in JavaScript** — Additional reference on safe query construction, used to validate the decision to use `replacements` on raw SQL endpoints.
<https://coreui.io/answers/how-to-prevent-sql-injection-in-javascript/>

Altered tables:

- **Why aren't tables being altered with sequelize.sync?** — Community answers explaining the index limit issue with `alter: true` and why switching to `force: false` is safer.
<https://stackoverflow.com/questions/43118777/why-arent-my-tables-being-altered-when-i-run-sequelize-sync>
- **beforeAll vs beforeEach in Jest** — Clarifies when to use each setup hook; used when structuring the test suite and managing database state between tests.
<https://stackoverflow.com/questions/54517032/beforeall-vs-beforeeach-when-to-use-them>

Jest:

- **Setup and teardown** — Documents `beforeAll`, `afterAll`, `beforeEach`, and `afterEach`; used to move `sequelize.sync()` into the correct lifecycle hook and close the connection cleanly with `afterAll`. <https://jestjs.io/docs/setup-teardown>

Sequelize upsert:

- **Sequelize upsert guide** — Practical explanation of `Model.upsert()` for insert-or-update logic; referenced when handling cart item deduplication.
<https://sebbastian.com/sequelize-upsert/>
-

Frontend

Vite:

- **Vite guide** — Getting started with Vite as a build tool and dev server; used when setting up the React project. <https://vite.dev/guide/>
- **Vite env variables** — How to define and access environment variables in Vite using `VITE_` prefixed keys; used for `VITE_API_URL`. <https://vite.dev/guide/env-and-mode>

Axios:

- **Axios instance** — How to create a custom Axios instance with a base URL and default headers; the basis for `axiosInstance.js`. <https://axios-http.com/docs/instance>
- **Axios interceptors** — How to attach request and response interceptors; used to automatically add the JWT to every request and handle 401 responses globally. <https://axios-http.com/docs/interceptors>

React:

- **createContext** — Creates a context object for sharing state across the component tree without prop drilling; used for `AuthContext` and `ToastContext`. <https://react.dev/reference/react/createContext>
- **useContext** — Hook for consuming a context value inside a component; used throughout pages to access auth state and toast functions. <https://react.dev/reference/react/useContext>
- **useState** — Hook for local component state; the basis for every page's data, loading, and modal state pattern. <https://react.dev/reference/react/useState>
- **useEffect** — Hook for side effects like data fetching and event listeners; used on every page to fetch data on mount and in the modal's keyboard listener. <https://react.dev/reference/react/useEffect>
- **useCallback** — Memoizes a function reference to prevent unnecessary re-renders; used to stabilize fetch functions passed as dependencies to `useEffect`. <https://react.dev/reference/react/useCallback>

React Router:

- **NavLink** — Like `Link` but adds an active class when the route matches; used in the sidebar for highlighted navigation. <https://reactrouter.com/en/main/components/nav-link>

- **useNavigate** — Hook that returns a function for programmatic navigation; used to redirect to `/login` on 401 and after logout. <https://reactrouter.com/en/main/hooks/use-navigate>
- **Navigate** — Declarative redirect component; used in `ProtectedRoute` to send unauthorized users to `/login`. <https://reactrouter.com/en/main/components/navigate>
- **Outlet** — Renders the matched child route inside a layout component; used in `Layout.jsx` to inject page content. <https://reactrouter.com/en/main/components/outlet>
- **useLocation** — Hook that returns the current location object; used to highlight the active sidebar link and handle post-login redirects. <https://reactrouter.com/en/main/hooks/use-location>

UI / Styling:

- **Lucide React** — Icon library used throughout the admin panel for sidebar icons, buttons, and status indicators. <https://lucide.dev/guide/packages/lucide-react>
- **Tailwind CSS docs** — Utility-first CSS framework used for all styling in the frontend. <https://tailwindcss.com/docs>

JavaScript (MDN):

- **Promise.allSettled()** — Runs multiple promises in parallel and returns all results regardless of whether they resolved or rejected; used on the dashboard to fetch all data concurrently without one failure blocking the rest. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/allSettled
- **Optional chaining (?.)** — Safely accesses nested object properties without throwing if an intermediate value is null or undefined; used extensively when reading nested backend response fields like `order?.User?.username`. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining
- **Nullish coalescing (??)** — Returns the right-hand value only when the left is `null` or `undefined`; used for safe fallback values in display logic. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing
- **localStorage** — Browser API for persisting key-value data across sessions; used to store and rehydrate the JWT token and user object. <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>

- **addEventListener** — Attaches an event handler to a DOM element or the window; used to listen for the `keydown` event to close modals on Escape.
<https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener>
- **KeyboardEvent** — Represents a keyboard interaction; used with `addEventListener` to detect when the user presses the Escape key.
<https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent>
- **Number.toFixed()** — Formats a number to a fixed number of decimal places; used when displaying prices and revenue totals in the UI.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toFixed
- **Computed property names** — Allows dynamic keys in object literals using bracket syntax; used when building form state objects where the field name is dynamic.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Object_initializer#computed_property_names

Charts

- **Recharts API** — Component reference for `BarChart`, `LineChart`, `XAxis`, `YAxis`, `Tooltip`, and related components used in the dashboard. <https://recharts.org/en-US/api>
- **Recharts npm** — Package page with version info and install instructions.
<https://www.npmjs.com/package/recharts>